

C++ Practice

Задание

Реализовать класс `Image`, который хранит изображение как динамический массив пикселей (`Pixel`).

Реализовать класс `TemplateImage`, который наследуется от `Image` и создаёт изображение, полностью заполненное чёрным цветом.

1. Реализовать класс `Pixel` (red, green, blue)
2. Реализовать `Image` с динамической памятью (хранит пиксели)
3. Реализовать Rule of 5 для `Image`
4. Добавить методы доступа к пикселям по координатам (добавить перегрузку оператора круглых скобок)
5. Перегрузить операторы (`+`, `==`, `<<`)
6. Использовать `static` и `const`
7. Реализовать `TemplateImage` с вызовом базового конструктора
8. В `main()` показать разницу `copy` vs `move`

Пояснения

1. Класс `Pixel`

Поля:

```
r, g, b;
```

Значение от 0 до 255

2. Класс `Image`

Поля:

```
int width;  
int height;  
Pixel* data;
```

Конструктор с параметрами:

```
Image(int w, int h);
```

Деструктор:

```
~Image();
```

Конструктор копирования:

```
Image(const Image& other);
```

Оператор копирующего присваивания:

```
Image& operator=(const Image& other);
```

Шаги:

- удалить старые данные
 - выделить новые
 - скопировать
-

Конструктор перемещения:

```
Image(Image&& other);
```

Передача указателя:

```
data = other.data;  
other.data = nullptr;
```

Оператор перемещающего присваивания:

```
Image& operator=(Image&& other);
```

3. Доступ к пикселям

```
Pixel& at(int x, int y);
```

Константная версия:

```
const Pixel& at(int x, int y) const;
```

Перегрузка круглых скобок:

```
const Pixel& operator()(int x, int y) const
```

4. Перегрузка операторов

Сложение изображений:

```
Image operator+(const Image& other) const;
```

```
image3 = image1 + image2;
```

Идея: сложить цвета пикселей ($200 + 100 = 255$ (max))

Сравнение:

```
bool operator\==\=(const Image& other) const;
```

Вывод:

```
std::ostream& operator<<(std::ostream& os, const Image& img);
```

5. Static

```
static int count;
```

Увеличивать в конструкторах, уменьшать в деструкторе (узнаем количество созданных изображений):

```
static int getCount();
```

```
Image -> count++
```

```
image.count
```

```
{  
image;  
}  
// count--;
```

6. Const методы

```
int getWidth() const; int getHeight() const;`
```

7. Класс TemplateImage

```
class TemplateImage : public Image
```

Конструктор:

```
TemplateImage(int w, int h);
```

Вызов базового:

```
TemplateImage(int w, int h) : Image(w, h)
```

Заполнение чёрным:

```
data[i] = {0, 0, 0};
```

8. Проверка (main)

- ☐ Копирование
- ☐ Перемещение
- ☐ Наглядно продемонстрировать разницу между копированием и перемещением
- ☐ Присваивание
- ☐ Работа с наследником

☐ Оператор сложения, вывода

// In Terminal

Inside Copy Constructor

Inside Move Constructor

// ...

```
git checkout main/master
git pull
git branch myname/task-image
git checkout myname/task-image
```

```
mkdir taskImage
// делат задание
```